

# Gold Market Dashboard

*How it was built: process, tools, and code*

This document walks through every step taken to turn the uploaded gold price history (gold\_historical\_data.csv), your existing Power BI dashboard (Gold\_Dashboard.pdf), and the reference layout (finance report screenshot) into an interactive HTML dashboard with a 3-month forecast — including the actual code used at each stage.

## 1. Overview of Tools Used

| Tool / Library           | What it was used for                                                                                                                        |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Python 3.12              | All data analysis, aggregation, and forecasting                                                                                             |
| pandas                   | Loading the CSV, date parsing, grouping/aggregating by year, quarter, month, and week                                                       |
| NumPy                    | Array operations and linear regression checks                                                                                               |
| statsmodels (ETSModel)   | Damped-trend exponential smoothing model for the 3-month price forecast, including 90% prediction intervals                                 |
| JSON                     | Packaging all computed results (KPIs, yearly/quarterly/monthly/weekly series, forecast) into a single data object embedded in the dashboard |
| HTML5 / CSS3             | Dashboard page structure, layout grid, card styling, color theming (CSS custom properties for easy re-theming)                              |
| JavaScript (vanilla)     | Wiring up the Year/Metric filter dropdowns and rebuilding charts on change                                                                  |
| Chart.js 4.4.0 (via CDN) | All charts: quarterly bar chart, monthly area chart, 10-year line chart, volume bar chart, forecast chart with confidence band              |
| docx (npm)               | Generating this Word document                                                                                                               |

## 2. Reviewing the Source Material

- Reference image (finance-style report): used only for layout pattern — header + icon, filter bar, KPI card row, chart panels below.
- Gold\_Dashboard.pdf: your existing Power BI dashboard — told me which metrics you already track (All-Time High/Low, price trend, volume trend) and the date range (2016-02-05 to 2026-01-30).
- gold\_historical\_data.csv: the raw daily data — Date, Open, High, Low, Close, Adj Close, Volume — 2,510 rows.

## 3. Loading and Exploring the Data

First pass: load the CSV, parse dates, and sanity-check ranges against the PDF.

```
import pandas as pd

df = pd.read_csv('gold_historical_data.csv', parse_dates=['Date'])
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Quarter'] = df['Date'].dt.quarter

print('High max:', df['High'].max())    # -> 5586.20    (matches PDF's 5.59K)
print('Low min:', df['Low'].min())      # -> 1123.90    (matches PDF's 1.12K)
print('Total volume:', df['Volume'].sum())
```

## 4. Aggregating the Data for Each Chart

Each chart in the dashboard needed a different time grouping. All aggregates were computed once in Python and exported as JSON.

### 4.1 Yearly summary

```
yearly = df.groupby('Year').agg(
    open=('Open', 'first'), close=('Close', 'last'),
    high=('High', 'max'), low=('Low', 'min'),
    avg=('Close', 'mean'), volume=('Volume', 'sum')
).reset_index()
```

### 4.2 Quarterly summary (for the bar chart)

```
quarterly = df.groupby(['Year', 'Quarter']).agg(
    close=('Close', 'last'), avg=('Close', 'mean'), volume=('Volume', 'sum')
).reset_index()
quarterly['label'] = quarterly['Year'].astype(str) + ' Q' + quarterly['Quarter'].astype(str)
```

### 4.3 Monthly summary (for the area chart)

```
df['YM'] = df['Date'].dt.strftime('%Y-%m')
monthly = df.groupby('YM').agg(
    close=('Close', 'last'), avg=('Close', 'mean'), volume=('Volume', 'sum')
).reset_index()
```

### 4.4 Weekly resample (for a smooth 10-year trend line)

```
weekly = df.set_index('Date').resample('W').agg(
    close=('Close', 'last'), volume=('Volume', 'sum')
).reset_index()
weekly['Date'] = weekly['Date'].dt.strftime('%Y-%m-%d')
```

## 5. Computing the KPI Cards

```
kpi = {
    'all_time_high': round(df['High'].max(), 2),
    'all_time_low': round(df['Low'].min(), 2),
    'latest_close': round(df['Close'].iloc[-1], 2),
    'latest_date': df['Date'].iloc[-1].strftime('%Y-%m-%d'),
    'total_volume': int(df['Volume'].sum()),
    'avg_close': round(df['Close'].mean(), 2),
    'records': len(df),
    'ytd_change_pct': round(
        (df['Close'].iloc[-1] /
         df[df['Year']==df['Year'].iloc[-1]]['Close'].iloc[0] - 1) * 100, 2)
}
```

## 6. Forecasting the Next 3 Months

For the forecast, I resampled to month-end closing prices and fit a damped-trend Exponential Smoothing (ETS) model — a standard method when a series shows strong momentum that is unlikely to continue accelerating forever. This produces both a point forecast and a 90% prediction interval.

```
from statsmodels.tsa.exponential_smoothing.ets import ETSModel

monthly_close = df.set_index('Date')['Close'].resample('ME').last()
monthly_close.index.freq = 'ME'
```

```

model = ETSModel(monthly_close, trend='add', damped_trend=True, seasonal=None)
fit = model.fit(dis=False)

pred = fit.get_prediction(start=len(monthly_close), end=len(monthly_close)+2)
summary = pred.summary_frame(alpha=0.10) # 90% interval
print(summary)

```

**Result (from the last actual close of \$4,713.90 on 2026-01-30):**

| Tool / Library | What it was used for                      |
|----------------|-------------------------------------------|
| Feb 2026       | \$4,880.62 (90% range: \$4,749 – \$5,012) |
| Mar 2026       | \$5,044.03 (90% range: \$4,843 – \$5,245) |
| Apr 2026       | \$5,204.17 (90% range: \$4,940 – \$5,469) |

*Caveat included directly in the dashboard: this is a statistical extrapolation of past price momentum only. It has no awareness of Fed policy, currency moves, central bank buying, or geopolitical events — all of which move gold more than trend continuation does.*

## 7. Building the Dashboard (HTML + CSS + JavaScript)

All computed results (KPIs + yearly/quarterly/monthly/weekly series + forecast) were packaged into one JSON object and embedded directly into a single self-contained .html file — no server or build step required.

```
import json

out = {
    'yearly': yearly.round(2).to_dict(orient='records'),
    'quarterly': quarterly.round(2).to_dict(orient='records'),
    'monthly': monthly.round(2).to_dict(orient='records'),
    'weekly': weekly.round(2).to_dict(orient='records'),
    'forecast': forecast_records,
    'kpi': kpi
}
data_json = json.dumps(out)

html = html_template.replace('__DATA_JSON__', data_json)
with open('gold_market_dashboard.html', 'w') as f:
    f.write(html)
```

### 7.1 Theming with CSS custom properties

Colors and fonts were defined as CSS variables, so switching the whole dashboard from dark to a white background later was a small, contained edit rather than rewriting every rule.

```
:root{
  --bg-deep:#f7f7f5;
  --bg-panel:#ffffff;
  --gold-1:#b8842e;
  --gold-2:#8a6218;
  --text-main:#1c1d1f;
  --text-dim:#767a82;
  --line:#e5e3dc;
}
```

### 7.2 Charts with Chart.js

Each panel is a <canvas> element driven by Chart.js. Example — the quarterly average price bar chart with a gold gradient fill:

```
quarterlyChart = new Chart(ctx, {
  type: 'bar',
  data: {
    labels: rows.map(d => d.label),
    datasets: [{
      label: 'Average',
      data: rows.map(d => d[metric]),
      backgroundColor: ctx => makeGradient(ctx, '#f0c04a', '#8a6218'),
      borderRadius: 6
    }]
  },
  options: { scales: { y: { ticks: { callback: v => '$' + v.toLocaleString() } } } }
});
```

### 7.3 Interactive filters

The Year and Metric dropdowns re-filter the underlying JSON arrays in the browser and rebuild the two charts — no server round-trip needed:

```
yearSel.addEventListener('change', refreshCharts);
metricSel.addEventListener('change', refreshCharts);

function refreshCharts(){
  quarterlyChart.destroy();
  monthlyChart.destroy();
  buildQuarterlyChart();
  buildMonthlyChart();
}
```

## 7.4 Forecast panel

The forecast chart plots actual monthly closes as a solid line, then bridges into a dashed red forecast line, with a shaded band showing the 90% interval (built from two overlapping datasets with fill: '+1' in Chart.js).

## 8. Iteration History

- v1 — Dark navy/gold themed dashboard matching the KPI-card + chart-panel layout from your reference image, populated with real CSV data (verified against your PDF).
- v2 — Added the 3-Month Price Forecast panel (chart, table, and caveats) using the ETS model above.
- v3 — Re-themed from dark to a white/cream background by updating CSS variables and the handful of chart colors Chart.js needs passed directly.

## 9. Where Everything Lives

- gold\_market\_dashboard.html — the final interactive dashboard (open in any browser, data is embedded, no internet needed except the Chart.js CDN link).
- All Python analysis was run in a sandboxed environment and is not saved as a separate file unless you'd like a copy — let me know and I can package the analysis script separately.